

DATA 260 | HOMEWORK 1

Rental Housing Listings

Web Development, Deployment, Agentic AI, and Model Evaluation

Pragya Apurva

August 31, 2026

0. Personal Configuration and Domain

The fixed configuration values are calculated from SID4 6102 and recorded here for reuse throughout the semester.

Value	Recorded result
SID4	6102
PORT_BASE	$8000 + (6102 \bmod 900) = 8702$
PREFIX	s6102
SEED	6102
VERIFY_SEED	$260000 + 6102 = 266102$
DOMAIN_ID	$6102 \bmod 8 = 6$
Assigned domain	Rental housing listings
Hardware	MacBook Air, Apple M4 (10-core), 24 GB memory
Local model	qwen3:1.7b through Ollama
GitHub repository	https://github.com/ishuapurva1996/data260-6102
Submission tag	hw1 (created after the final local verification)

1. Domain Schema

DOMAIN_SCHEMA.md defines the entity before implementation. The entity is a rental housing listing with the following fields and category values.

Field	Control	Required	Definition / allowed values
listingTitle	Text input	Yes	Primary field; title of the rental listing
propertyAddress	Text input	Yes	Secondary field; rental property address
submitterEmail	Email input	Yes	Email address of the person submitting the listing
description	Textarea	Yes	Detailed property description; must exceed 25 characters
propertyType	Dropdown	Yes	apartment, house, condo, townhouse
termsAccepted	Checkbox	Yes	I agree to the terms and conditions.

2. HTML and CSS Implementation

The HTML page uses the title HW1-Pragya_Apurva, an h1 heading for the assigned domain, required form controls, autofocus on the primary field, four domain-specific categories, the required terms label, and a script element at the end of the body.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HW1-Pragya_Apurva</title>
  <link rel="stylesheet" href="css_style.css">
</head>
<body>
  <h1>Rental Housing Listings</h1>
  <form id="rentalForm">
    <label for="listingTitle">Listing Title <sup>*</sup></label>
    <input type="text" id="listingTitle" name="listingTitle" placeholder="ex: Zia Apartment" required
autofocus>
    <br><br>
    <label for="propertyAddress">Address <sup>*</sup></label>
    <input type="text" id="propertyAddress" name="propertyAddress" placeholder="ex: 999 Main Street, CA"
required>
    <br><br>
    <label for="submitterEmail">Landlord Email <sup>*</sup></label>
    <input type="email" id="submitterEmail" name="submitterEmail" placeholder="landlord@email.com"
required>
    <br><br>
    <label for="description">Description <sup>*</sup></label>
    <textarea name="description" id="description" placeholder="e.g., Spacious two-bedroom apartment with
parking"
      rows="3" cols="40" required></textarea>
    <br><br>
    <label for="propertyType">Property Type</label>
    <select name="propertyType" id="propertyType" required>
      <option value="" selected disabled>Choose</option>
      <option value="apartment">apartment</option>
      <option value="house">house</option>
      <option value="condo">condo</option>
      <option value="townhouse">townhouse</option>
    </select>
    <br><br>
    <input type="checkbox" id="termsAccepted" name="termsAccepted">
    <label for="termsAccepted">I agree to the terms and conditions.</label>
    <br><br>
    <button type="submit">Create Rental Listing</button>
  </form>
  <h2>Submitted Rental Listings</h2>
  <ul id="rentalList"></ul>
  <script src="script_hw.js"></script>
</body>
</html>
```

Rendered output

Figure 1. Rental Housing Listings form served from the Docker container at localhost:8702.

CSS presentation

The CSS centers the form, keeps controls inside their containers, adapts to smaller screens, and presents submitted listings as readable cards with a green accent and a red Delete button.

```
body {
  font-family: Arial, sans-serif;
  background-color: rgb(237, 237, 237);
  margin: 0;
  padding: 50px 24px;
}

form {
  background-color: white;
  width: 100%;
  max-width: 710px; margin: 30px auto;
  padding: 39px 55px; box-sizing: border-box;
  border-radius: 12px;
  box-shadow: 0 2px 10px gray;
}

input[type="text"], input[type="email"], textarea, select {
  width: 100%;
  padding: 10px;
  box-sizing: border-box;
  border: 2px solid rgb(223, 220, 220);
  border-radius: 5px;
}
```

3. JavaScript Validation

The arrow function trims the relevant values and returns false when the description is 25 characters or fewer or when the terms checkbox is not selected. Each failure displays the required alert.

```
const validateForm = () => {
  const listingTitle = document.getElementById("listingTitle").value.trim();
  const propertyAddress = document.getElementById("propertyAddress").value.trim();
  const submitterEmail = document.getElementById("submitterEmail").value.trim();
  const description = document.getElementById("description").value.trim();
  const termsAccepted = document.getElementById("termsAccepted");

  if (!listingTitle || !propertyAddress || !submitterEmail || !description) {
    alert("All fields are required!");
    return false;
  }
  if (description.length <= 25) {
    alert("Description must contain more than 25 characters.");
    return false;
  }
  if (!termsAccepted.checked) {
    alert("Please agree to the terms and conditions.");
    return false;
  }
  return true;
};
```

Validation output A: short description

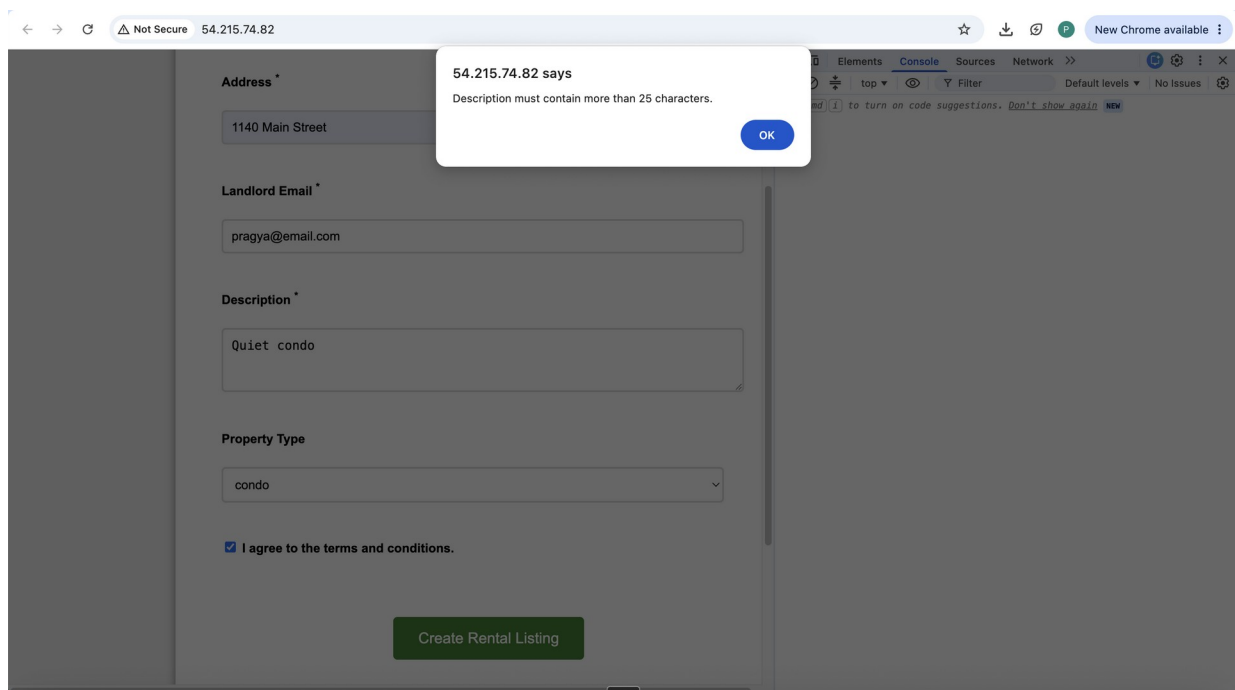


Figure 2. A description with 25 or fewer characters is rejected with the required alert.

Validation output B: terms not accepted

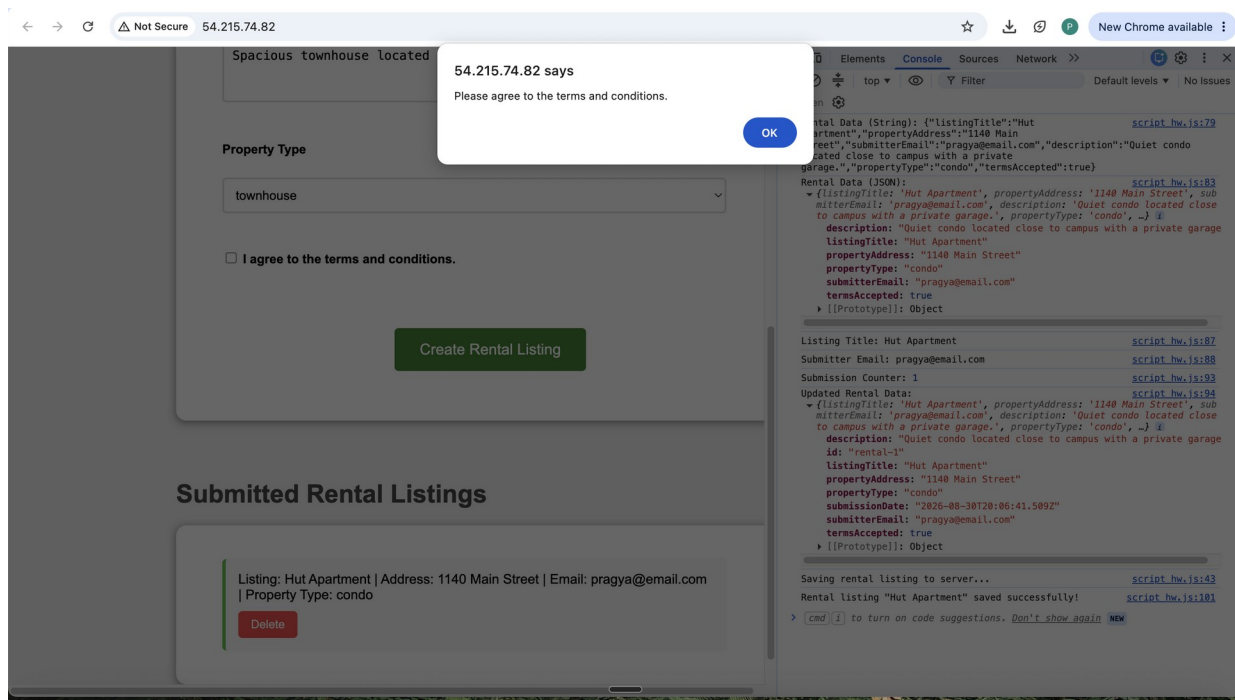


Figure 3. A valid description with an unchecked terms box is rejected with the required alert.

4. JSON Processing, Destructuring, Spread, and Closure

After validation, the handler serializes the form data, parses the JSON, destructures the primary and email fields, adds submissionDate with the spread operator, and advances a closure-based counter for each successful submission.

```
const submissionCounter = (() => {
  let count = 0;
  return () => ++count;
})();

const rentalData = {
  listingTitle, propertyAddress, submitterEmail,
  description, propertyType, termsAccepted
};

const jsonRentalData = JSON.stringify(rentalData);
console.log("Rental Data (String):", jsonRentalData);

const parsedRentalData = JSON.parse(jsonRentalData);
const {
  listingTitle: rentalListingTitle,
  submitterEmail: landlordEmail
} = parsedRentalData;
console.log("Listing Title:", rentalListingTitle);
console.log("Submitter Email:", landlordEmail);

const currentCount = submissionCounter();
const updatedRentalData = {
  ...parsedRentalData,
  submissionDate: new Date().toISOString(),
  id: `rental-${currentCount}`
};
console.log("Submission Counter:", currentCount);
console.log("Updated Rental Data:", updatedRentalData);
```

Successful output

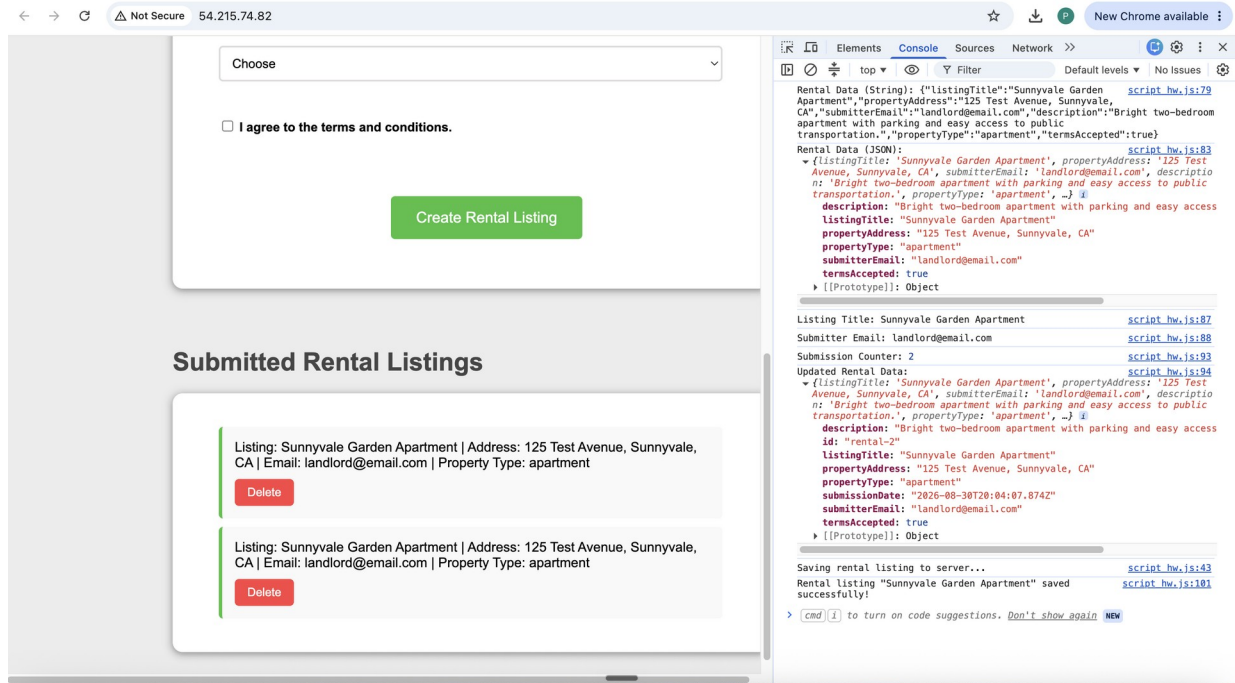


Figure 4. Two successful submissions produce two listing cards and Submission Counter: 2 in the console.

Observed result: The console shows the JSON string, parsed object, destructured title and email, updated object with submissionDate, and an incrementing closure count. The visible list and Delete buttons are additional UI behavior.

Storage behavior: No backend database or localStorage is used. Submitted cards exist only in the current page DOM and disappear after a refresh. The simulated server Promise always resolves after two seconds.

5. Docker Deployment

The application is packaged in an Nginx Alpine image. The course port 8702 on the host maps to Nginx port 80 in the container.

```
FROM nginx:alpine

COPY web_application/index_hw1.html /usr/share/nginx/html/index.html
COPY web_application/css_style.css /usr/share/nginx/html/css_style.css
COPY web_application/script_hw.js /usr/share/nginx/html/script_hw.js

EXPOSE 80
```

Reproducible Docker commands

```
# Run from repository root
docker build --platform linux/amd64 -t rental-housing-app:latest code
docker run -d --name rental-housing-hw1 -p 8702:80 rental-housing-app:latest
docker ps
open http://localhost:8702
```

The linux/amd64 image matches the ECS X86_64 task definition. Docker Desktop emulates that architecture on the Apple Silicon development machine.

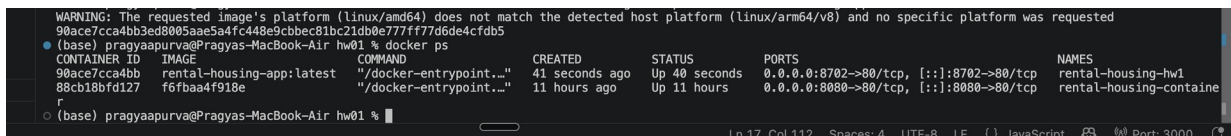


Figure 5. Docker confirms that rental-housing-hw1 is running with host port 8702 mapped to container port 80.

Docker localhost output

The screenshot shows a web browser window with the address bar displaying 'localhost:8702'. The page contains a form for creating a rental listing and a section for submitted listings.

Description*

e.g., Spacious two-bedroom apartment with parking

Property Type

Choose

☐ I agree to the terms and conditions.

Create Rental Listing

Submitted Rental Listings

Listing: Sunnyvale valley Apartment | Address: 1140 Main Street | Email: pragya@email.com | Property Type: apartment

Delete

Figure 6. Successful listing output from the Docker-hosted application at localhost:8702.

6. AWS ECS Deployment

The same linux/amd64 image was pushed to a private Amazon ECR repository and deployed as a one-task ECS service using AWS Fargate in us-west-1.

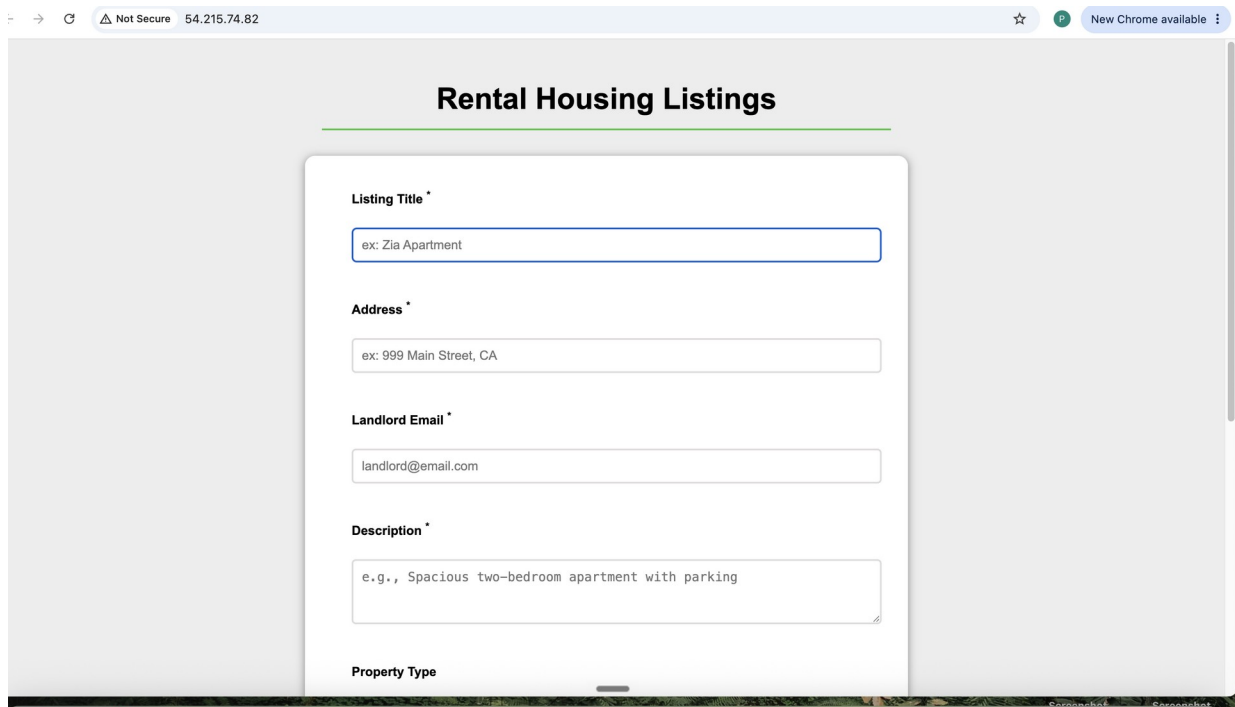
Resource	Recorded configuration
Region	us-west-1 (N. California)
ECR repository	rental-housing-app
Image URI	463984225910.dkr.ecr.us-west-1.amazonaws.com/rental-housing-app:latest
ECS cluster	rental-housing-app-cluster
Task definition	rental-housing-app-task:1; Fargate; Linux/X86_64; 0.25 vCPU; 0.5 GB
Container	rental-housing-app-container; TCP port 80; essential
ECS service	rental-housing-app-service; desired tasks: 1
Security group	rental-housing-app-sg; inbound HTTP TCP/80
Public address during test	http://54.215.74.82 (temporary task IP)

The task received a public IP and used the custom security group to allow inbound HTTP traffic. A load balancer was not needed for this one-task homework deployment.

The screenshot shows the AWS ECS console for the 'rental-housing-app-cluster'. The service 'rental-housing-app-service' is in an 'Active' state. The task list shows one task with ID '7bcb668a65014222a659...' in a 'Running' state. The container details for this task show the container name 'rental-housing-app-conta...', runtime ID '7bcb668a650142...', image URI '46398422...', and image digest 'sha256:f8...'. The container is in a 'Running' state with a health status of 'Unknown' and 0 CPU usage.

Figure 7. ECS service is Active with one desired task, one running task, zero pending tasks, and a successful deployment.

Public-IP application output



The screenshot shows a web browser window with the address bar displaying 'Not Secure' and the IP address '54.215.74.82'. The page title is 'Rental Housing Listings'. The form contains the following fields:

- Listing Title ***: Input field with placeholder text 'ex: Zia Apartment'.
- Address ***: Input field with placeholder text 'ex: 999 Main Street, CA'.
- Landlord Email ***: Input field with placeholder text 'landlord@email.com'.
- Description ***: Textarea with placeholder text 'e.g., Spacious two-bedroom apartment with parking'.
- Property Type**: Input field (partially visible).

Figure 8. Rental Housing Listings application running from the ECS task public IP address.

Network note: The browser reports Not Secure because the homework deployment uses HTTP on port 80 without a domain name or TLS certificate. The ECS task public IP can change when the task is replaced.

7. Part 1 Test Results

Test	Input / action	Expected result	Result
Valid submission	All fields valid; description >25 characters; terms checked	Listing appears; required console logs print; counter increments	PASS
Short description	Description = Quiet condo	Alert states description must exceed 25 characters; no new listing	PASS
Terms unchecked	Valid long description; checkbox unchecked	Terms alert appears; no new listing	PASS
Closure	Two valid submissions without refresh	Console counts 1 then 2	PASS
Docker	Open localhost:8702	Application loads from Nginx container	PASS
AWS ECS	Open task public IP over HTTP	Application loads; service maintains one running task	PASS

8. Part 1 Completion Checklist

Requirement	Status
DOMAIN_SCHEMA.md defines fields and categories	COMPLETE
Required HTML form and script link	COMPLETE
Arrow-function validation with both alerts	COMPLETE
JSON string, parse, destructuring, and spread output	COMPLETE
Closure logs successful-submission count	COMPLETE
Docker image built and application run on PORT_BASE	COMPLETE
AWS ECS service runs exactly one task	COMPLETE
Localhost and public-IP screenshots included	COMPLETE

9. Agentic AI (Part 2)

A local Planner and Reviewer communicate through Ollama. The Reviewer receives the Planner's raw response, and only the deterministic Python Finalizer normalizes and publishes the final JSON.

Run setting	Recorded value
Domain	Rental Housing Listings (DOMAIN_ID 6)
Python	3.12.13
Ollama	0.33.0
Model	qwen3:1.7b
Temperature	0.0

Model choice

The requested qwen3:8b download repeatedly stalled near 79 percent. I documented the substitution and used qwen3:1.7b, which fits the available hardware and completed the required local runs.

Exact command

```
python code/agents_demo.py --title "Modern Two-Bedroom Apartment Near Downtown San Jose" --content "Bright two-bedroom apartment with in-unit laundry, covered parking, pet-friendly policies, and convenient light rail access near downtown San Jose." --model qwen3:1.7b --temperature 0.0
```

Short answers

Q1. Final tags: modern two-bedroom apartment; pet-friendly; light rail access.

Q2. Final summary: Modern two-bedroom apartment near downtown San Jose, pet-friendly, and light rail access.

Q3. Yes. The Reviewer kept the three tags but shortened the Planner's phrase 'with pet-friendly policies' to the more concise 'pet-friendly.'

10. How the Pipeline Works

1. Input

The program accepts a title and content. No rental-domain keywords are embedded in the agent logic.

2. Planner

The first adapter call asks the Planner LLM for three tags and a one-sentence summary as strict JSON.

3. Raw handoff

The exact Planner response is placed in the Reviewer conversation without Python repair.

4. Reviewer

The second adapter call checks relevance, uniqueness, and the 25-word limit and returns a corrected raw proposal.

5. Python Finalizer

Python parses and normalizes only the Reviewer result, rejects unresolved issues, and publishes exactly three input-derived tags plus the summary.

```
Planner LLM call
  -> raw Planner response
Reviewer LLM call
  -> raw Reviewer response
Python validation and Finalizer
  -> publish JSON
```

11. Planner and Reviewer: Code and Output

The exact command below invokes the two local-model calls. Its recorded Terminal output follows immediately.

```
python code/agents_demo.py --title "Modern Two-Bedroom Apartment Near Downtown San Jose" --content "Bright two-bedroom apartment with in-unit laundry, covered parking, pet-friendly policies, and convenient light rail access near downtown San Jose." --model qwen3:1.7b --temperature 0.0

--- Planner (5939 ms) ---
{
  "thought": "Identified three relevant tags: 'Modern Two-Bedroom Apartment,' 'Pet-Friendly,' and 'Light Rail Access.' Summary focuses on key features and location. Issues: summary exceeds 25 words, tags may overlap.",
  "message": "Modern two-bedroom apartment near downtown San Jose with pet-friendly policies and light rail access.",
  "data": {
    "tags": [
      "modern two-bedroom apartment",
      "pet-friendly",
      "light rail access"
    ],
    "summary": "Modern two-bedroom apartment near downtown San Jose with pet-friendly policies and light rail access.",
    "issues": []
  }
}

--- Reviewer (8574 ms) ---
{
  "thought": "Identified three distinct tags: 'Modern Two-Bedroom Apartment,' 'Pet-Friendly,' and 'Light Rail Access.' Summary focuses on key features and location. Issues: summary exceeds 25 words.",
  "message": "Modern two-bedroom apartment near downtown San Jose, pet-friendly, and light rail access.",
  "data": {
    "tags": [
      "modern two-bedroom apartment",
      "pet-friendly",
      "light rail access"
    ],
    "summary": "Modern two-bedroom apartment near downtown San Jose, pet-friendly, and light rail access.",
    "issues": []
  }
}

(base) pragyapurva@Pragyas-MacBook-Air hw01 %
```

Figure 9. Genuine Terminal output from the recorded Planner and Reviewer run.

12. Finalizer and Publish: Code and Output

```
result = run_pipeline(model, args.title, args.content, args.email)
print("--- Finalized Output ---")
print(json.dumps(result["finalized"], indent=2, ensure_ascii=False))
print("--- Publish Output ---")
print(json.dumps(result["package"], indent=2, ensure_ascii=False))
```

```
--- Finalized Output ---
{
  "thought": "Reviewer proposal selected and schema validated.",
  "message": "Final tags and summary are ready for publishing.",
  "data": {
    "tags": [
      "modern two-bedroom apartment",
      "pet-friendly",
      "light rail access"
    ],
    "summary": "Modern two-bedroom apartment near downtown San Jose, pet-friendly, and light rail access.",
    "issues": []
  }
}
(base) pragyaapurva@Pragyas-MacBook-Air hw01 %
```

Figure 10a. Genuine Terminal output from the deterministic Finalizer.

```
--- Publish Output ---
{
  "title": "Modern Two-Bedroom Apartment Near Downtown San Jose",
  "email": "student@example.com",
  "reviewerChanged": true,
  "final": {
    "tags": [
      "modern two-bedroom apartment",
      "pet-friendly",
      "light rail access"
    ],
    "summary": "Modern two-bedroom apartment near downtown San Jose, pet-friendly, and light rail access.",
    "issues": []
  },
  "submissionDate": "2026-08-30T23:32:41.597977Z"
}
(base) pragyaapurva@Pragyas-MacBook-Air hw01 %
```

Figure 10b. Genuine Terminal output of the validated publish JSON.

13. Measuring Non-Determinism (Part 3)

The fixed title and content were saved once in `cases/nondeterminism_input.json`, then used unchanged for 20 successful runs at temperature 0.7 and 20 at temperature 0.0.

Fixed input

Title: Modern Two-Bedroom Apartment Near Downtown San Jose

Content: Bright two-bedroom apartment with in-unit laundry, covered parking, pet-friendly policies, and convenient light rail access near downtown San Jose.

Tag-set results

Metric	Temperature 0.7	Temperature 0.0
Distinct tag sets	12	2
Tags in all 20 runs	None	modern two-bedroom apartment; pet-friendly
Tags in exactly one run	8 tags: convenient light rail access; downtown san jose light rail; in-unit laundry & light rail; modern; modern apartment; near downtown san jose; pet-friendly and light rail access; pet- friendly with parking	light rail access

Latency results

Metric (ms)	Temperature 0.7	Temperature 0.0
p50	22,820.50	18,844.50
p95	48,035.70	25,745.40
p99	54,126.34	26,025.08

Interpretation

At temperature 0.7, two users can receive different but still topical emphases, such as downtown location, pet-friendly policies, or light rail. At temperature 0.0, 19 of 20 runs produced the same tag set, so the behavior was much more stable.

Variation is acceptable for exploratory tags or search suggestions because several faithful descriptions can be useful. It is not acceptable when output controls eligibility, compliance, or billing, where identical inputs should produce identical decisions.

Attempt accounting

Temperature 0.7 needed 22 attempts to obtain 20 publishable runs; the Finalizer rejected two attempts with unresolved Reviewer issues. Temperature 0.0 completed 20 successful runs in 20 attempts. Rejected attempts remain in the raw JSON for auditability.

Recorded command and output

```
python reports/hw01/run_nondeterminism.py --model qwen3:1.7b --count 20 --temperatures 0.7 0.0
--timeout 120 --max-errors 10
```

```
{
  "0.7": {
    "runs_requested": 20,
    "attempts": 22,
    "successful_runs": 20,
    "failed_attempts": 2,
    "distinct_tag_sets": 12,
    "tags_in_all_successful_runs": [],
    "tags_in_exactly_one_run": [
      "convenient light rail access",
      "downtown san jose light rail",
      "in-unit laundry & light rail",
      "modern",
      "modern apartment",
      "near downtown san jose",
      "pet-friendly and light rail access",
      "pet-friendly with parking"
    ],
    "latency_ms": {
      "p50": 22820.5,
      "p95": 48035.7,
      "p99": 54126.34
    }
  },
  "0.0": {
    "runs_requested": 20,
    "attempts": 20,
    "successful_runs": 20,
    "failed_attempts": 0,
    "distinct_tag_sets": 2,
    "tags_in_all_successful_runs": [
      "modern two-bedroom apartment",
      "pet-friendly"
    ],
    "tags_in_exactly_one_run": [
      "light rail access"
    ],
    "latency_ms": {
      "p50": 18844.5,
      "p95": 25745.4,
      "p99": 26025.08
    }
  }
}
(base) pragyaapurva@Pragyas-MacBook-Air hw01 %
```

Figure 11. Genuine Terminal output for both 20-run temperature experiments.

14. Model Client and Token Accounting (Part 4)

All model calls now go through `src/model_client.py`. The adapter provides one stable `complete(messages, tools=None)` interface, while `/stats` reads counters without adding messages to the conversation history.

Reusable adapter

```
class OllamaModelClient:
    def complete(self, messages, tools=None):
        target = self._model.bind_tools(list(tools)) if tools else self._model
        response = target.invoke(list(messages))
        usage = _token_usage(response)
        self.stats.turn_count += 1
        self.stats.input_tokens += usage.input_tokens
        self.stats.output_tokens += usage.output_tokens
        return CompletionResult(response.content, usage, response)
```

AGENT.md instruction

- Return only Markdown bullet points.
- Start every non-empty line with '- '.
- Do not use headings, numbered lists, introductions, or closing remarks.
- Focus on correctness, edge cases, readability, and the smallest useful fix.

Verification result: all five local-model responses passed the bullet-only check.

Recorded command

```
python code/hw1_client.py --demo --model qwen3:1.7b --temperature 0.0 --timeout 120 --output
reports/hw01/raw/part4_five_turn_conversation.txt --metrics-output
reports/hw01/raw/part4_token_counts.json
```

15. Five-Turn Conversation Evidence

```
for turn, prompt in enumerate(DEMO_PROMPTS, start=1):
    record = run_turn(client, system_prompt, history, prompt)
    if turn in {3, 5}:
        print_stats(history, client.stats)
```

```
DATA 260 HW1 - Part 4 Model Client Demo
Run timestamp (UTC): 2026-08-31T04:11:27.558449Z
Model: qwen3:1.7b
Temperature: 0.0

--- Turn 1 ---
User: Review this Python function for correctness: def average(values): return sum(values) / len(values)
Assistant:
- The function is correct in its basic logic but lacks a check for empty input. If `values` is empty, it will raise a division by zero error.
Tokens: input=115, output=34, total=149
Bullet-only check: PASS

--- Turn 2 ---
User: Now focus only on the empty-list edge case.
Assistant:
- If `values` is an empty list, the function will raise a `ZeroDivisionError` because dividing by zero is not allowed.
Tokens: input=168, output=29, total=197
Bullet-only check: PASS

--- Turn 3 ---
User: Suggest one minimal fix without changing the function's purpose.
Assistant:
- Add a check to ensure `values` is not empty before calculating the average.
Tokens: input=218, output=18, total=236
Bullet-only check: PASS

/stats
{
  "turn_count": 3,
  "cumulative_input_tokens": 501,
  "cumulative_output_tokens": 81,
  "cumulative_total_tokens": 582,
  "serialized_history_characters": 731
}
(base) pragyapurva@Pragyas-MacBook-Air hw01 %
```

Figure 12. Genuine Terminal output for turns 1-3, per-turn token counts, and the first /stats snapshot.

15. Five-Turn Conversation Evidence (continued)

```
print("--- Cumulative usage on exit ---")
print(f"turns={client.stats.turn_count}, input_tokens={client.stats.input_tokens}, "
      f"output_tokens={client.stats.output_tokens}, total_tokens={client.stats.total_tokens}")
```

```
--- Turn 4 ---
User: Review this revision: def average(values): return None if not values else sum(values) / len(values)
Assistant:
- The function now returns `None` when `values` is empty, which avoids a `ZeroDivisionError`.
Tokens: input=266, output=24, total=290
Bullet-only check: PASS

--- Turn 5 ---
User: Give a final two-bullet assessment covering behavior and readability.
Assistant:
- **Behavior**: The function now correctly returns `None` when `values` is empty, avoiding a `ZeroDivisionError`.
- **Readability**: The code is more explicit about the empty list case, improving clarity and maintaining intent.
Tokens: input=311, output=50, total=361
Bullet-only check: PASS

/stats
{
  "turn_count": 5,
  "cumulative_input_tokens": 1078,
  "cumulative_output_tokens": 155,
  "cumulative_total_tokens": 1233,
  "serialized_history_characters": 1348
}

--- Cumulative usage on exit ---
turns=5, input_tokens=1078, output_tokens=155, total_tokens=1233
AGENT.md bullet-only verification: PASS
(base) pragyapurva@Pragyas-MacBook-Air hw01 %
```

Figure 13. Genuine Terminal output for turns 4-5, final /stats, cumulative usage, and bullet-only verification.

Per-turn tokens

Turn	Input	Output	Total	Bullet-only
1	115	34	149	PASS
2	168	29	197	PASS
3	218	18	236	PASS
4	266	24	290	PASS
5	311	50	361	PASS

/stats snapshots

Snapshot	Turns	Input	Output	History characters
After turn 3	3	501	81	731
After turn 5	5	1078	155	1348

16. Why the Token Counts Grow

Why resend prior context?

Model calls are stateless. The client must resend earlier messages so the next response can use the conversation.

System prompt versus user message

The system prompt defines the role and conversation-wide rules. A user message contains the current request.

Why input tokens grow

Every later request includes a longer conversation history, so more text is tokenized and sent to the model.

What limits the growth

The model has a finite context window. Older content eventually has to be trimmed or summarized to leave room for the new request and response.

17. AI Assistant Use

1. What I used it for

I used an AI assistant to audit the repository against the assignment, suggest targeted fixes and tests, and help organize the report. I supplied the configuration and run evidence, checked suggestions against the rubric, and decided what to submit.

2. One item I independently verified

The audit flagged box-sizing: box-block in the CSS. I verified that it is invalid, so browsers ignored it and form controls could overflow on narrow screens.

3. How I verified it

I inspected the source and loaded the page at 1280 px and 390 px widths. I then reran Node syntax validation and all 46 automated tests.

4. What I changed

I changed it to border-box and added responsive width and max-width rules. Controls now remain inside the form, which stays centered on desktop and fits mobile without horizontal overflow.

18. Reproducibility and Verification

The exact commands are listed in README.md and RUN_LOG.txt. The complete local self-check is:

```
make -C reports/hw01 verify-hw01 PYTHON=python NODE_BIN=node
```

Check	Expected result
Python	Version 3.11 or 3.12
Part 1	Required HTML controls and valid JavaScript syntax
Automated tests	All Homework 1 tests pass
Part 3	20 successful runs at each temperature
Part 4	Five turns, token totals, /stats snapshots, bullet-only PASS
Report	Valid PDF in reports/hw01/report.pdf

Submission note: the repository is submitted at the hw1 tag. The PDF uploaded to the course must match reports/hw01/report.pdf stored at that tagged commit.

Tagged GitHub repository: <https://github.com/ishuapurva1996/data260-6102/tree/hw1>